

Argussight & MXCuBE

Multi-camera video integration – architecture & configuration

SOLEIL PROXIMA-2 · July 3, 2026

Contents

1 Overview	1
2 Architecture: the three communication paths	2
2.1 Path 1 – Discovery (backend to Argussight, gRPC 50051)	2
2.2 Path 2 – Video (browser to Argussight, WebSocket 7000)	2
2.3 Path 3 – Input (cameras to Argussight, Redis)	3
2.4 The whole picture	3
2.5 Consequences of this design	3
3 Inside Argussight	4
3.1 How a Streamer produces a stream	4
3.2 The GetProcesses response	4
4 How MXCuBE integrates Argussight	4
4.1 Backend – discovery	4
4.2 Frontend – the camera selector	5
4.3 Snapshots and calibration	5
5 Configuration (step by step)	5
5.1 The video-streamer	5
5.2 The Argussight server	6
5.3 The MXCuBE side	8
5.4 Bring-up checklist	8
5.5 Troubleshooting	9
6 Reference	9
6.1 Ports	9
6.2 MXCuBE server.yaml keys	9
6.3 video-streamer flags	10

1 Overview

Argussight is a standalone video-processing and aggregation service, originally built as an extension of MXCuBE's [video-streamer](#). It runs as its **own process, on its own host**, independent of MXCuBE. Its job is to take one or more camera sources and expose them as a set of uniform, browser-ready **MPEG1 WebSocket streams** behind a single proxy port.

MXCuBE PROXIMA-2 uses Argussight to make **four physical beamline cameras** available in the web UI – including the on-axis viewer (OAV) used for sample centring. The main sample-view pane becomes switchable between the four cameras; the centring overlay (click-to-centre, beam marker, shapes) is enabled only when the OAV camera is selected.

The integration is deliberately **loosely coupled**:

- MXCuBE’s backend makes exactly one small remote call to Argussight (to *discover* which streams exist). It never proxies video.
- The heavy video traffic flows **directly from the browser to Argussight**, so the MXCuBE server is never a video bottleneck.
- If Argussight is unreachable, MXCuBE still starts: it simply falls back to the OAV camera’s direct video-streamer URL.

This document explains how the pieces talk to each other, how MXCuBE integrates Argussight, and – in Section 5 – exactly how to configure the video-streamer and Argussight for a four-camera beamline.

2 Architecture: the three communication paths

Argussight exposes two network ports, used by two *different* parts of MXCuBE, plus a third, upstream path by which it acquires the video in the first place. Keeping these three paths separate is the key to understanding the system.

Port	Protocol	Who connects	Purpose
50051	gRPC	MXCuBE backend (Flask)	control / discovery of streams
7000	WebSocket	the browser (each pane)	the live video pixels (MPEG1)
6379	Redis	cameras / video-streamer	frame input into Argussight

2.1 Path 1 – Discovery (backend to Argussight, gRPC 50051)

When the browser loads the sample view, it asks the MXCuBE backend for the sample-image metadata. That triggers a single gRPC `GetProcesses` call from the Flask backend to Argussight, which returns the list of available stream names. The backend turns each name into a WebSocket URL and hands the list to the frontend.

```

MXCuBE backend (Flask)                                Argussight
-----
discover_streams() ----- GetProcesses ---> gRPC :50051
                                <----- [oav, cam2, ... ] --
                                build ws://host:7000/ws/<name> for each

```

This is the **only** time MXCuBE’s Python talks to Argussight – a short request/response, fully guarded against failure.

2.2 Path 2 – Video (browser to Argussight, WebSocket 7000)

The backend does not carry any video; it only tells the browser the URLs. Each video pane in the browser then opens a WebSocket **directly** to Argussight’s stream proxy and decodes the MPEG1 frames with JSMpeg onto a canvas.

```

Browser (JSMpeg)                                Argussight proxy
-----
new JSMpeg.Player(                                WS :7000
  "ws://host:7000/ws/oav") =====> /ws/oav   (MPEG1 frames)

```

```
switch camera in dropdown --> tear down,      /ws/cam2
reconnect to /ws/cam2      =====> ...
```

MXCuBE's backend is not in this loop at all.

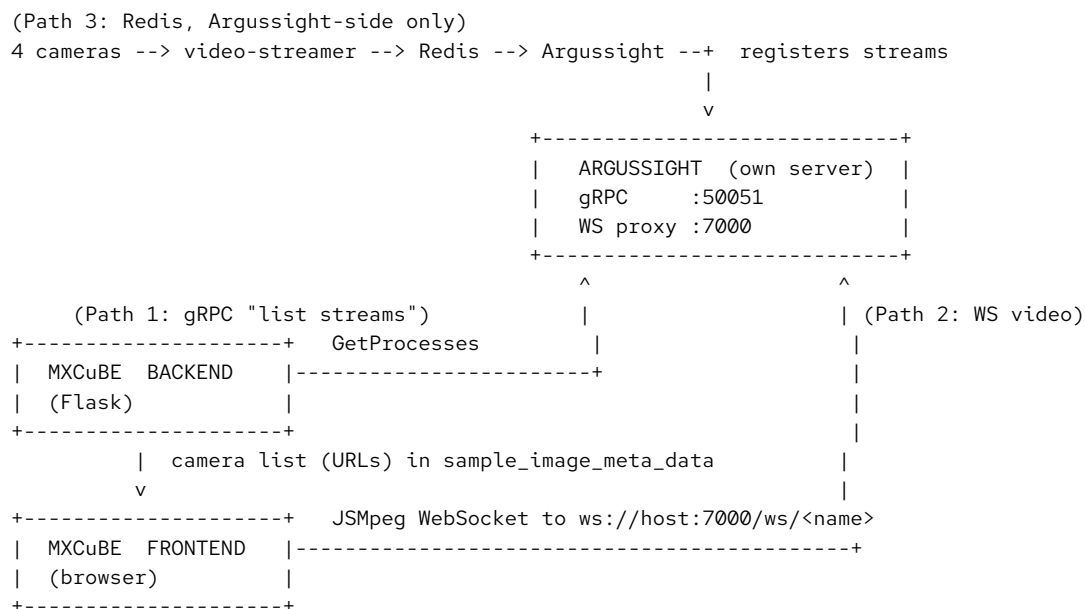
2.3 Path 3 – Input (cameras to Argussight, Redis)

This path is upstream of MXCuBE entirely: it is how Argussight *obtains* the video. A camera's frames are published to Redis; the video-streamer (or an internal Argussight streamer process) reads them, encodes MPEG1, and the stream is registered on the proxy under a name.

```
physical camera --> video-streamer --> Redis --> Argussight streamer --> :7000/ws/<name>
                    (encodes MPEG1)   (frames)   (re-publishes + registers)
```

MXCuBE plays no part in this path; you configure it on the Argussight side (Section 5).

2.4 The whole picture



2.5 Consequences of this design

- **Loose coupling.** The backend makes one small gRPC call. Argussight owns the stream lifecycle.
- **Graceful fallback.** If Argussight is down, discovery returns an empty list and the OAV keeps its direct video-streamer URL – MXCuBE still works.
- **Scalable video.** Video goes browser-to-Argussight directly; the MXCuBE server is never a relay.
- **Reachability matters.** The browser must reach Argussight on port 7000, and the backend must reach it on 50051. On networks where the browser and backend see Argussight under different hostnames, these two addresses differ (see 5.3).
- **Snapshots stay local.** Sample snapshots and the pixels-per-mm / beam-position calibration still come from MXCuBE's RedisMpegVideo camera object – Argussight supplies only the *live* video.

3 Inside Argussight

Argussight is built from four cooperating components:

- **Spawner** – the core; starts, tracks and terminates video processes, and owns the connection to the stream proxy.
- **gRPC server** – the control surface (port 50051); receives `GetProcesses`, `AddStream`, `StartProcesses`, etc.
- **Video processes** – each runs one video task. A process that inherits the `Streamer` class produces a stream.
- **Stream-layer (proxy)** – an abstraction layer (port 7000) that re-exposes every registered stream at `ws://host:7000/ws/<name>`, hiding the real source port.

3.1 How a Streamer produces a stream

A `Streamer` process subscribes to a Redis channel of raw frames, processes each frame, re-publishes the result to its own Redis id, and launches a `video-streamer` subprocess that serves MPEG1 on a free port (starting at `streams_starting_port`, default 9000). The Spawner registers that port + id with the proxy under the process's unique name. From the outside, everything is reached uniformly at `ws://host:7000/ws/<name>`.

Streams can also be **registered from outside** Argussight (any existing MPEG1 / MJPEG `video-streamer`) through the `AddStream` gRPC call – this is the simplest way to expose four already-running camera streamers (see 5.2).

3.2 The `GetProcesses` response

`GetProcesses` returns a status, the running processes, the available process types and – the field MXCuBE uses – the list of registered **stream names**:

```
GetProcessesResponse {
  status: "success"
  running_processes: { ... }
  available_process_types: [ ... ]
  streams: ["oav", "hutch", "cryo", "sample_changer"]
}
```

4 How MXCuBE integrates Argussight

The integration lives entirely in the MXCuBE web layer (`mxweb`); no changes were needed in `mxcore`. Discovery is surfaced through the existing `sample-view` metadata endpoint that the sample view already consumes.

4.1 Backend – discovery

`mxweb/core/util/argussight_discovery.py` performs the guarded gRPC call:

- imports the Argussight gRPC stubs lazily (missing stubs -> warning + empty list);
- opens an insecure channel to `ARGUSSIGHT_GRPC_HOST:ARGUSSIGHT_GRPC_PORT`, calls `GetProcesses` with a short timeout;
- for each stream name builds `{name, label, url, format, width, height, oav}`, with `url = ARGUSSIGHT_PROXY_URL/<name>`;

- if ARGUSSIGHT_CAMERAS is configured, restricts and orders the result to those names (and applies their labels / sizes / oav flag);
- returns [] on **any** error, so an unreachable Argussight never breaks the UI.

mxweb/core/adapters/sample_view_adapter.py calls this from sample_image_meta_data(). When streams are found it adds a cameras list to the payload and points the default videoURL at the OAV stream (with an empty videoHash, since Argussight URLs already carry the stream name).

The new settings live on MXCUBEAppConfigModel (mxweb/core/models/configmodels.py): ARGUSSIGHT_ENABLED, ARGUSSIGHT_GRPC_HOST, ARGUSSIGHT_GRPC_PORT, ARGUSSIGHT_PROXY_URL and ARGUSSIGHT_CAMERAS (a list of _ArgussightCameraModel, each with an optional oav flag marking the centring camera).

4.2 Frontend – the camera selector

In ui/src/components/SampleView/SampleImage.jsx:

- an overlay **dropdown** (top-left of the video) lists the discovered cameras and dispatches selectCamera(name);
- the JSMpeg player is re-initialised whenever the stream URL changes, so switching cameras tears down the old player and connects to the new one;
- the video source is built from the complete URL when videoHash is empty (Argussight), or the legacy URL/hash form otherwise;
- the FabricJS **centring overlay, click-to-centre and go-to-beam are disabled unless the OAV camera is selected** (centringEnabled).

The Redux sampleview slice gains cameras, selectedCamera and centringEnabled, plus a SELECT_CAMERA action that repoints the player and recomputes centringEnabled from the chosen camera's oav flag.

4.3 Snapshots and calibration

The OAV's RedisMpegVideo hardware object in mxweb/core is still required: it serves get_last_image() for snapshots and the diffractometer's pixels-per-mm / beam position for centring. Argussight only supplies the four *live* video streams.

5 Configuration (step by step)

This section is the practical core: how to configure the video-streamer, the Argussight server, and the MXCuBE side, for a four-camera beamline.

5.1 The video-streamer

The video-streamer process reads frames from a source (typically a Redis buffer that a camera fills) and serves them as an MPEG1 (or MJPEG) stream over an HTTP/WebSocket port. One video-streamer instance = one output stream.

Its command-line flags:

Flag	Meaning
-uri	source URI, e.g. redis://localhost:6379 or test
-hs	host the stream server binds to
-p	port the stream is served on
-q	JPEG quality (lower = better, e.g. 4)
-s	frame size as WIDTH,HEIGHT
-of	output format: MPEG1 or MJPEG
-id	unique stream id (hash) used in the WebSocket path
-irc	input Redis key/channel to read frames from

On PROXIMA-2 the OAV camera object `RedisMpegVideo` launches a video-streamer for you. Its YAML (the real example is `singleton_objects/camera.yaml` / `md_camera.yaml` in the web config repo) looks like:

```
class: mxcube.core.HardwareObjects.RedisMpegVideo.RedisMpegVideo
configuration:
  uri: redis://localhost:6379
  host: <camera-host>
  port: 8000
  format: MPEG1
  width: 1360
  height: 1024
  quality: 10
  redis_key: mxcube.web
```

From those properties it spawns, effectively:

```
video-streamer -uri redis://localhost:6379 -hs <camera-host> -p 8000 \
-q 10 -s 1360,1024 -of MPEG1 -id <stream-hash> -irc mxcube.web
```

To test a video-streamer by itself – without any camera – use the built-in test source, which produces a synthetic MPEG1 stream:

```
video-streamer -uri test -hs localhost -p 8010 -of MPEG1 -id teststream
```

Then point a browser / JSMpeg client at `ws://localhost:8010/ws/teststream`.

One video-streamer per camera. For four cameras you run four instances, each on its own port, each reading its own camera's Redis buffer, each with its own `-id`.

5.2 The Argussight server

Install (in the Argussight host's environment). Argussight needs `ffmpeg` on the system and is installed with `pip install .` (or `poetry install`) from the repository.

Configure `argussight/core/configurations/config.yaml`:

Key	Meaning
modules_path	Python package holding the process classes
worker_classes	available process types (name -> class, accessible)
processes	processes started automatically on boot
wait_time	seconds to wait for a process before killing it
streams_layer_port	the proxy port clients connect to (default 7000)
streams_starting_port	first port used when a Streamer picks a free port

The Redis source that streamer processes read from is set when launching the server: `--host / -hs`, `--port / -p`, and `--channel / -ch` (default channel video-streamer).

Start the server:

```
argussight -hs localhost -p 6379 -ch video-streamer
```

This brings up the gRPC server on 50051 and the stream proxy on 7000.

Register the four cameras. There are two ways to get streams onto the proxy:

1. *External streams (recommended when you already run video-streamers).* Start each camera's video-streamer (Section 5.1), then register it with Argussight via the AddStream gRPC call, giving it a **name**. That name is what MXCuBE discovers and what appears in `ws://host:7000/ws/<name>`.

```
import grpc, uuid, subprocess
import argussight.grpc.argus_service_pb2 as pb2
import argussight.grpc.argus_service_pb2_grpc as pb2_grpc

channel = grpc.insecure_channel("localhost:50051")
stub = pb2_grpc.SpawnerServiceStub(channel)

# for each camera: a running video-streamer on its own port + id ...
port = "8000"; sid = str(uuid.uuid1())
# subprocess.Popen(["video-streamer", "-uri", "...", "-p", port,
#                  "-of", "MPEG1", "-id", sid, "-irc", "<redis-key>"])

# ... then expose it on the proxy under a stable name:
stub.AddStream(pb2.AddStreamRequest(name="oav", port=port, stream_id=sid))
```

1. *Internal Streamer processes.* Write/enable process classes that inherit Streamer in `config.yaml`; the Spawner starts them, spawns their video-streamer, and registers them automatically under their configured name.

Verify what is registered:

```
import grpc
import argussight.grpc.argus_service_pb2 as pb2
import argussight.grpc.argus_service_pb2_grpc as pb2_grpc

stub = pb2_grpc.SpawnerServiceStub(grpc.insecure_channel("localhost:50051"))
resp = stub.GetProcesses(pb2.GetProcessesRequest())
print(resp.status, list(resp.streams)) # -> success ['oav', 'hutch', 'cryo', ...]
```

The names printed here are exactly the names to use in MXCuBE's ARGUSSIGHT_CAMERAS.

5.3 The MXCuBE side

Edit the `mxcube:` section of the web server config (`webconfig/mxcube-web/server.yaml`) and enable Argussight:

```
ARGUSSIGHT_ENABLED: true
ARGUSSIGHT_GRPC_HOST: <argussight-host>
ARGUSSIGHT_GRPC_PORT: 50051
ARGUSSIGHT_PROXY_URL: ws://<argussight-host>:7000/ws
ARGUSSIGHT_CAMERAS:
- { name: oav,    label: OAV (centring), width: 1360, height: 1024, oav: true }
- { name: hutch, label: Hutch,           width: 800,  height: 450 }
- { name: cryo,  label: Cryo,            width: 800,  height: 450 }
- { name: sc,    label: Sample changer, width: 800,  height: 450 }
```

Rules and notes:

- Each name **must match** the stream name registered in Argussight (5.2). A name that does not match is silently dropped from the selector.
- Exactly **one** camera should have `oav:true`. The sample view starts on it and only it gets the centring overlay.
- `ARGUSSIGHT_PROXY_URL` is dialled by the **browser**, so use a hostname the browser can reach. `ARGUSSIGHT_GRPC_HOST` is dialled by the **backend**. On a split network these two may differ.
- Leave `ARGUSSIGHT_CAMERAS` empty to expose **every** discovered stream (unfiltered, in discovery order) – useful for a first smoke test.
- The MXCuBE web environment must have Argussight's gRPC stubs importable: `pip install argussight` (or vendor `argus_service_pb2.py` and `argus_service_pb2_grpc.py`). If absent, discovery logs a warning and the OAV falls back to its direct `VIDEO_STREAM_URL`.
- Keep the OAV RedisMpegVideo hardware object configured – it still provides snapshots and centring calibration (4.3).

5.4 Bring-up checklist

1. Redis is running and each camera fills its own Redis buffer/key.
2. One video-streamer per camera is running (or will be started at registration), each on its own port with its own `-id`.
3. The Argussight server is running (`argussight ...`); gRPC 50051 and proxy 7000 are reachable.
4. The four streams are registered (`AddStream`) with stable names; `GetProcesses` lists all four.
5. `server.yaml` has `ARGUSSIGHT_ENABLED:true`, the correct host/proxy URL, and `ARGUSSIGHT_CAMERAS` names matching step 4 (one with `oav:true`).
6. The MXCuBE web environment can import the Argussight gRPC stubs.
7. Start MXCuBE. The sample view shows a camera dropdown with the four cameras; switching plays each live MPEG1 stream; centring works only on the OAV.

5.5 Troubleshooting

Symptom	Likely cause / fix
No dropdown, only the OAV shows	Discovery returned empty: Argussight down, wrong gRPC host/port, or stubs not installed. Check the backend log for the discovery warning.
A configured camera is missing	Its name does not match a registered stream. Compare with GetProcesses.
Dropdown lists cameras but panes are black	Browser cannot reach the proxy. Check ARGUSSIGHT_PROXY_URL host/port from the client, and that the stream is actually publishing.
Centring disabled on the right camera	The <code>oav: true</code> flag is on the wrong entry, or on none.
Backend hangs briefly at login	gRPC host wrong/firewalled; the call times out. Fix the host or firewall; discovery then fails fast to [].

6 Reference

6.1 Ports

Port	Service	Consumer
6379	Redis	cameras, streamers
8000+	per-camera video-streamer (internal)	Argussight proxy
9000+	Argussight internal streamer output	Argussight proxy
50051	Argussight gRPC	MXCuBE backend
7000	Argussight stream proxy	the browser

6.2 MXCuBE `server.yaml` keys

Key	Default	Meaning
ARGUSSIGHT_ENABLED	false	turn discovery on
ARGUSSIGHT_GRPC_HOST	localhost	Argussight gRPC host (backend dials this)
ARGUSSIGHT_GRPC_PORT	50051	Argussight gRPC port
ARGUSSIGHT_PROXY_URL	(empty)	base WS URL (browser dials this)
ARGUSSIGHT_CAMERAS	(empty)	per-stream label/size/oav; empty = all

6.3 video-streamer flags

Flag	Meaning
-uri	source URI (redis://... or test)
-hs	bind host
-p	serve port
-q	JPEG quality
-s	WIDTH,HEIGHT
-of	MPEG1 or MJPEG
-id	stream id (path hash)
-irc	input Redis key/channel